

¿Cómo funciona Begiz

s los procesos de la app explicados: qué haces tú, qué pasa por dentro y dónde queda guardado · 1 de octubre de 20

Contenido

1. Las piezas de la app

2. Iniciar sesión

3. Guardar una web

4. Subir o cambiar la imagen

5. El mosaico y los filtros
6. Detalle, edición y borrado

7. Dónde se guarda cada dato

8. Seguridad

9. Cómo trabajamos

10. Publicación y costes

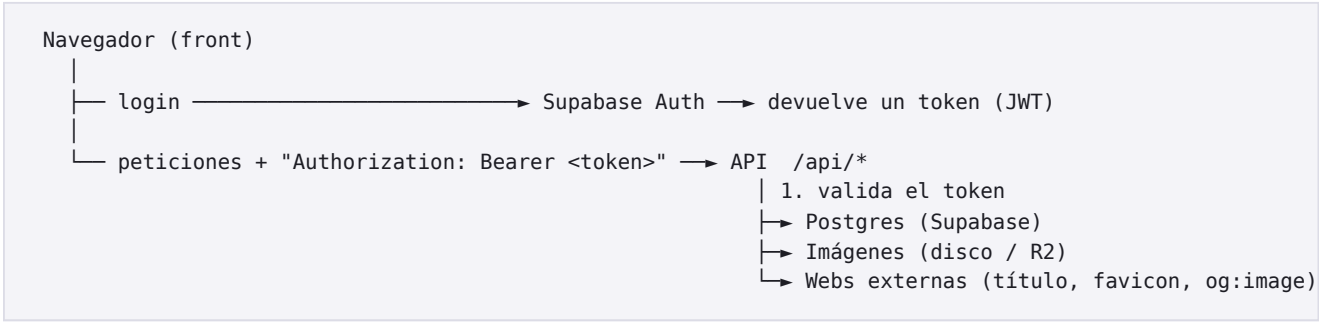
11. Hoja de ruta

1. Las piezas de la app

Begiz se compone de cinco piezas. Tú solo ves la primera; el resto trabaja por detrás.

Pieza	Dónde está	Qué hace
Front	apps/web · Vue 3 + Vite	La app que ves en el navegador: el mosaico, los formularios, los filtros. No guarda datos: se los pide a la API.
API	apps/api · Hono + Drizzle	Un programa en un servidor que recibe las peticiones del front, comprueba quién eres, lee las páginas que guardas, procesa las imágenes y habla con la base de datos.
Supabase Auth	Tu proyecto de Supabase	Gestiona las cuentas: email, contraseña cifrada y las sesiones.
Base de datos	Postgres en Supabase	Guarda las webs, tags y colecciones de cada usuario.
Imágenes	Disco local / Cloudflare R2	Guarda los archivos de imagen. En la base de datos solo queda la ruta de cada uno.

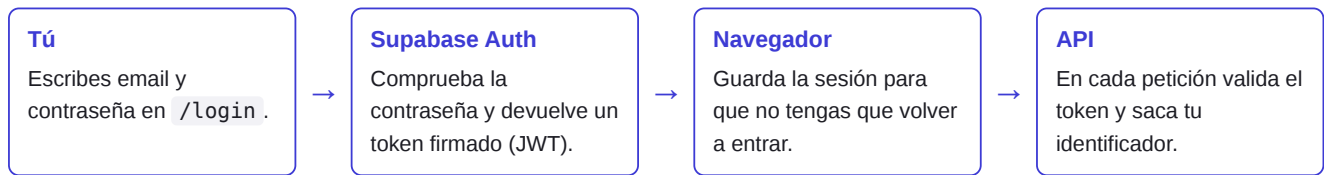
Además hay un paquete compartido, `packages/shared`, con las reglas de validación (qué es una URL válida, cuánto puede medir un título, cómo se normaliza un tag). Lo usan el front y la API, así que las reglas son siempre las mismas en los dos lados.



La regla de oro

El front nunca toca la base de datos. Todo pasa por la API, que es la única que sabe las reglas. Así la futura extensión del navegador podrá usar la misma API sin repetir nada.

2. Iniciar sesión



- **El token** es como un pase firmado por Supabase que dice «esta persona es la usuaria X» y caduca pasado un tiempo. La API comprueba la firma con las claves públicas de tu proyecto (`apps/api/src/auth.ts`), así que nadie puede fabricar uno falso.
- Si el token falta, es falso o ha caducado, la API responde **401** y no devuelve ningún dato. La librería de Supabase renueva el token sola antes de que caduque, así que en el uso normal no lo notas.
- El **registro público está desactivado**: las cuentas solo se crean desde el panel de Supabase.
- El router del front (`apps/web/src/router.ts`) también te manda al login si intentas abrir una página sin sesión, y al volver te lleva a donde ibas.

3. Guardar una web

Es el proceso más completo de la app. Empieza en `/new` (`NewWebView.vue`) y lo resuelve la ruta `POST /api/webs` (`apps/api/src/routes/webs.ts`).

#	Quién	Qué pasa
1	Tú	Rellenas enlace, título, notas, tags y colecciones. Opcionalmente pegas, arrastras o eliges una imagen.
2	Front	Envía los datos a la API. Si has puesto imagen propia, le dice que <i>no</i> descargue la de la página (<code>usePageImage: false</code>).
3	API	Valida los datos con las reglas de <code>packages/shared</code> : URL <code>http/https</code> , título no vacío (máx. 200), notas (máx. 5000), hasta 30 tags de 40 caracteres. Los tags se normalizan: « Dark Mode » pasa a «dark mode», y los repetidos se quitan.
4	API	Lee la página con protección SSRF (ver sección 8): título (<code>og:title</code> o <code><title></code>), favicon y <code>og:image</code> . Si la página no responde, no pasa nada: la web se guarda igual, sin esos datos.
5	API	Si toca usar la imagen de la página, la descarga y crea la miniatura WebP (<code>createThumbnail</code> , en <code>lib/preview.ts</code>) y la guarda en el almacenamiento. Si falla, la web queda sin imagen.
6	Base de datos	En una sola transacción guarda la web, crea los tags que no existían, los enlaza y añade la web a sus colecciones. Si algo falla (p. ej. una colección que no es tuya), no se guarda nada y se borra la miniatura que se acababa de crear.
7	Front	Si elegiste imagen propia, ahora la sube (ver sección 4). Después refresca el mosaico, los tags y los contadores, y te lleva al detalle de la web.

¿Qué es una transacción?

Un grupo de operaciones en la base de datos que se hacen todas o ninguna. Así nunca queda una web a medias, por ejemplo guardada pero sin sus tags.

Prioridad de la imagen de vista previa

Orden	Fuente
1	Tu imagen , subida a mano. Puede ser una captura de página completa.
2	La <code>og:image</code> de la página: la imagen que muestran las webs al compartir un enlace. Se copia a nuestro almacenamiento para que no se rompa si la web la cambia.
3	Una tarjeta generada con el favicon y el dominio (<code>WebThumb.vue</code>). El mosaico nunca tiene huecos.

4. Subir o cambiar la imagen

Ocurre al guardar una web con imagen propia o con «Change image» en el detalle. Ruta: `PUT /api/webs/:id/preview` .

1. **Tú** pegas una imagen con Ctrl+V (funciona en cualquier parte de la página), la arrastras o la eliges del disco (`ImagePicker.vue`).
2. La **API comprueba que la web es tuya**; si no, responde 404.

3. **Valida el contenido real** del archivo con la librería `sharp ( detectUploadFormat )`. No se fía de la extensión ni de lo que dice el navegador. Acepta PNG, JPG, WebP, GIF y AVIF, hasta 25 MB.
4. **Crea la miniatura**: WebP de 800 px de ancho. Si la imagen es muy alta (una captura de página completa), la miniatura se recorta a los 1200 px de arriba.
5. **Guarda dos archivos**: la miniatura (para el mosaico) y el original (para el detalle, a tamaño completo).
6. Actualiza la web en la base de datos y **borra las imágenes anteriores**, para no acumular archivos huérfanos.

Cada archivo lleva un identificador aleatorio nuevo en su nombre, así que una imagen nunca se sobrescribe: se crea otra y se borra la vieja. Esto permite que el navegador las guarde en caché para siempre.

5. El mosaico y los filtros

La página principal (`HomeView.vue`) pide las webs a `GET /api/webs` con los filtros activos.

Filtro	En la URL	Cómo funciona
Buscar por título	<code>?search=portfolio</code>	Busca el texto dentro del título, sin distinguir mayúsculas. Los caracteres especiales (<code>%</code> , <code>_</code>) se tratan como texto normal. Un índice especial (<code>pg_trgm</code>) mantiene la búsqueda rápida aunque haya miles de webs.
Tags	<code>?tags=dark,serif</code>	Devuelve solo las webs que tienen todos los tags elegidos (Y), que es lo útil para buscar inspiración concreta.
Colección	<code>?collection=<id></code>	Solo las webs de esa colección. Se combina con los demás filtros.

- **Los filtros viven en la URL:** si recargas, compartes el enlace o vas hacia atrás, se mantienen.
- El buscador espera **250 ms** después de la última tecla antes de pedir resultados, para no lanzar una petición por cada letra.
- Las webs salen **de la más nueva a la más antigua**.
- **Caché (TanStack Query):** el front recuerda las respuestas de la API. Al volver a una página ya vista, la muestra al instante. Al cambiar un filtro, mantiene el resultado anterior en pantalla hasta que llega el nuevo, sin parpadeos. Después de guardar, editar o borrar, invalida esa caché para que todo se vuelva a pedir actualizado.
- La lista de tags del filtro (`GET /api/tags`) muestra solo los que están en uso, **ordenados por cuántas webs los usan**. La misma lista sirve para el autocompletado al escribir tags.

6. Detalle, edición y borrado

Ver y editar una web

- El detalle (`/webs/:id`) muestra la imagen a tamaño completo si existe, el enlace a la web, el título original de la página y el formulario de edición.
- Al guardar cambios (`PATCH /api/webs/:id`) solo se actualiza lo que envías. Los **tags se reemplazan** por la lista nueva, y los que ya no usa ninguna web **se borran solos**. Las **colecciones también se reemplazan**, comprobando que todas son tuyas.

Qué se borra y qué se conserva

Acción	Se borra	Se conserva
Borrar una web	La web, sus enlaces con tags y colecciones, sus imágenes y los tags que quedan sin uso.	Las colecciones (con una web menos) y los tags que usan otras webs.
Borrar una colección	La colección y la lista de sus webs.	Todas las webs. Solo dejan de pertenecer a esa colección.
Quitar un tag de una web	El enlace; y el tag, si ya no lo usa ninguna web.	La web y el resto de sus tags.

Colecciones

- Se crean desde el formulario de guardar o editar una web (`CollectionPicker.vue`).

- En `/collections` se listan con su número de webs y se pueden renombrar o borrar.
- No puede haber dos colecciones con el mismo nombre: la API responde **409** con un mensaje que se muestra en pantalla.

7. Dónde se guarda cada dato

Dato	Dónde	Detalles
Cuenta	Supabase Auth Panel → Authentication → Users	Email y contraseña cifrada. Cada usuario tiene un identificador único (uuid).
Webs	Tabla <code>webs</code>	Enlace, título, notas, título y favicon de la página, rutas de la miniatura y del original, origen de la imagen y fechas.
Tags	Tablas <code>tags</code> y <code>web_tags</code>	Los nombres de los tags y qué tags tiene cada web. Un nombre no se repite dentro de la misma usuario.
Colecciones	Tablas <code>collections</code> y <code>collection_webs</code>	Nombre y descripción de cada colección, y qué webs contiene.
Imágenes	Desarrollo: <code>apps/api/.data/uploads/</code> Producción: Cloudflare R2	Organizadas como <code><usuario>/<web>/<id>.webp</code> . La base de datos solo guarda la ruta.
Sesión	Tu navegador	Solo el token de login. Ningún dato de la app se guarda en el navegador.

Todas las usuarias comparten las mismas tablas. Cada fila lleva una columna `user_id` con su dueña, y la API filtra por ella en **todas** las consultas.

Ojo con las imágenes en local

Mientras se trabaja en local, las imágenes solo existen en tu ordenador, y la carpeta `.data` no se sube a GitHub. Si se borra o cambias de equipo, las webs siguen en Supabase pero salen con la tarjeta de favicon. Al publicar, estas imágenes no pasan solas a R2: habrá que copiarlas si se quieren conservar.

8. Seguridad

Riesgo	Cómo se evita
Ver datos de otra persona	Cada consulta filtra por tu <code>user_id</code> . Si pides una web o colección que no es tuya, la API responde 404 como si no existiera, y no puedes meter tus webs en colecciones ajenas. Los tests de aislamiento lo comprueban en cada cambio.
Acceso directo a la base de datos	Supabase permite leer tablas con la clave pública que usa el front. Por eso todas las tablas tienen RLS (Row Level Security) activado y sin permisos: ese acceso queda cerrado. La API se conecta como propietaria y no le afecta.
SSRF	La API abre las URLs que escribes. Alguien podría usarla para alcanzar direcciones internas del servidor (<code>localhost</code> , redes privadas, metadatos del proveedor cloud). <code>lib/safe-fetch.ts</code> bloquea las direcciones no públicas en el momento de conectar y en cada redirección, con un tiempo máximo de 8 s y un tamaño máximo de descarga.
Archivos maliciosos	Las imágenes se validan por su contenido real, no por la extensión. Un archivo que no es imagen se rechaza aunque se llame <code>.png</code> .
Imágenes públicas	Se sirven sin login, pero sus rutas llevan identificadores aleatorios imposibles de adivinar.
Cuentas ajenas	Registro público desactivado. Solo existen las cuentas creadas desde el panel.

9. Cómo trabajamos

Comandos

Comando	Para qué
<code>pnpm dev</code>	Arranca el front (<code>localhost:5173</code>) y la API (<code>localhost:8787</code>). Los cambios en el código se recargan solos.
<code>pnpm test</code>	Ejecuta los tests de la API y de <code>shared</code> . Unos 25 segundos, sin Supabase ni internet.
<code>pnpm typecheck</code>	Comprueba los tipos de TypeScript en todo el proyecto.
<code>pnpm build</code>	Genera la versión de producción del front.
<code>pnpm db:generate</code>	Crea una migración después de cambiar el esquema (<code>apps/api/src/db/schema.ts</code>).
<code>pnpm db:migrate</code>	Aplica las migraciones pendientes a la base de datos de Supabase.

Los tests

- Usan **PGLite**, un Postgres de verdad que se ejecuta en memoria y se crea con las mismas migraciones. Cada test empieza con la base de datos vacía y nunca toca tu Supabase.
- Firman tokens igual que Supabase, así que **pasan por la validación real del login**.
- Las páginas externas se **simulan** (`test/fake-web.ts`): los tests no dependen de que una web real esté disponible.
- Cubren login, guardado, filtros, edición, borrado, imágenes, colecciones, aislamiento entre usuarias y SSRF: 53 tests en la API y 11 en `shared` .

Cambiar la base de datos

1. Se edita el esquema en `apps/api/src/db/schema.ts` .
2. `pnpm db:generate` crea un archivo SQL con el cambio en `apps/api/drizzle/` . Toda tabla nueva lleva RLS activado.
3. `pnpm test` : los tests crean su base de datos con esa migración, así que un error sale aquí antes de tocar Supabase.
4. `pnpm db:migrate` la aplica en Supabase. Las migraciones aplicadas ya no se editan nunca.

Antes de dar algo por terminado

Verificación mínima

`pnpm test` y `pnpm typecheck` en verde (y `pnpm build` si se toca el front). Cada cambio en la API lleva sus tests. Los commits se hacen cuando tú lo pides, con mensajes en inglés.

10. Publicación y costes

Pieza	Servicio	Gratis hasta	Cuándo se paga
Front	Cloudflare Pages (estático)	Prácticamente sin límite a esta escala	Nunca, para este uso
API	Railway, Fly.io o Render (Node)	Sin plan gratuito útil (el de Render se duerme y tarda 30–60 s en despertar)	~0–5 \$/mes desde el principio
Base de datos + login	Supabase	500 MB (cientos de miles de webs)	Pro, 25 \$/mes, solo si hiciera falta
Imágenes	Cloudflare R2	10 GB, descargas siempre gratis	~0,015 \$ por GB y mes por encima de 10 GB

- **El límite real son las imágenes.** Con miniaturas de `og:image` (50–150 KB) caben unas 100.000 webs en 10 GB. Con capturas de página completa subidas a mano (2–5 MB cada una), unas 2.000–5.000.
- **Supabase gratuito se pausa** tras 7 días sin uso. Se reactiva desde el panel sin perder datos.
- **Sin copias de seguridad descargables** en el plan gratuito: al publicar, conviene preparar una exportación periódica.

Cifras del plan de arquitectura (septiembre de 2026). Los proveedores cambian sus planes; se revisarán antes de publicar.

11. Hoja de ruta

Fase	Contenido	Estado
1 · Base	Login, webs, tags, colecciones, búsqueda, vista previa por <code>og:image</code> o imagen manual	Validada
1b · Tests	Vitest, tests de la API y de <code>shared</code> ; código revisado (inglés, nombres, comentarios)	Hecha
Diseño	Diseño visual de la plataforma	Siguiente
2 · Captura	Captura automática con Playwright (necesita un servidor con Chromium, ~5 \$/mes)	Planificada
3 · Extensión	Extensión del navegador para guardar con un clic, usando la misma API	Planificada
4 · Componentes	Guardar partes concretas de una web con su imagen, tags y notas	Futuro
Idiomas	Selector de idioma: inglés (por defecto) y español	Futuro
Tema	Cambio entre tema claro y oscuro	Futuro

Para el detalle técnico de cada decisión: `ARCHITECTURE.md`. Para poner la app en marcha: `README.md`.